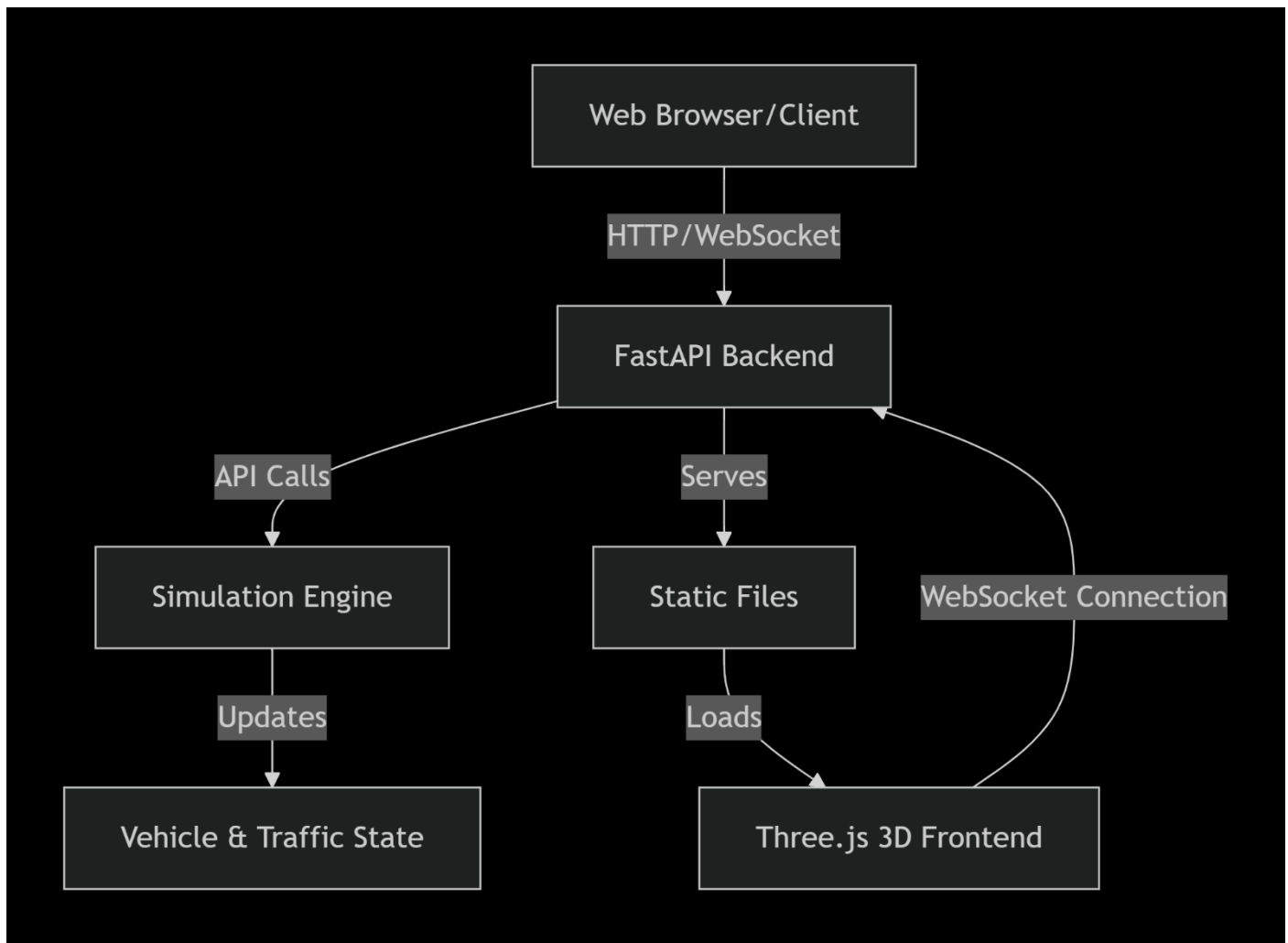


1. System Architecture Overview



The system follows a Client-Server Architecture with real-time updates through WebSocket connections.

2. Programming Methods

Object-Oriented Programming (OOP)

- Simulation Engine class (simulation.py)
- Pydantic Models (schemas.py)
- Modular separation of concerns into different files (city_generator, pathfinding, simulation)

Functional Programming Elements

- Pure functions for city generation (generate_city(), make_building())
- Immutable data structures where possible (using dictionaries and lists)
- Pipeline pattern for data transformation

Asynchronous Programming

- WebSocket handler (@app.websocket('/ws'))
- Async/await pattern for handling concurrent connections

3. Backend Frameworks & Libraries

Core Framework

- FastAPI
- Uvicorn: ASGI server for running the FastAPI application

Data Validation & Serialization

- Pydantic: Schema definition for all API responses
- Python typing

4. Frontend Frameworks & Libraries

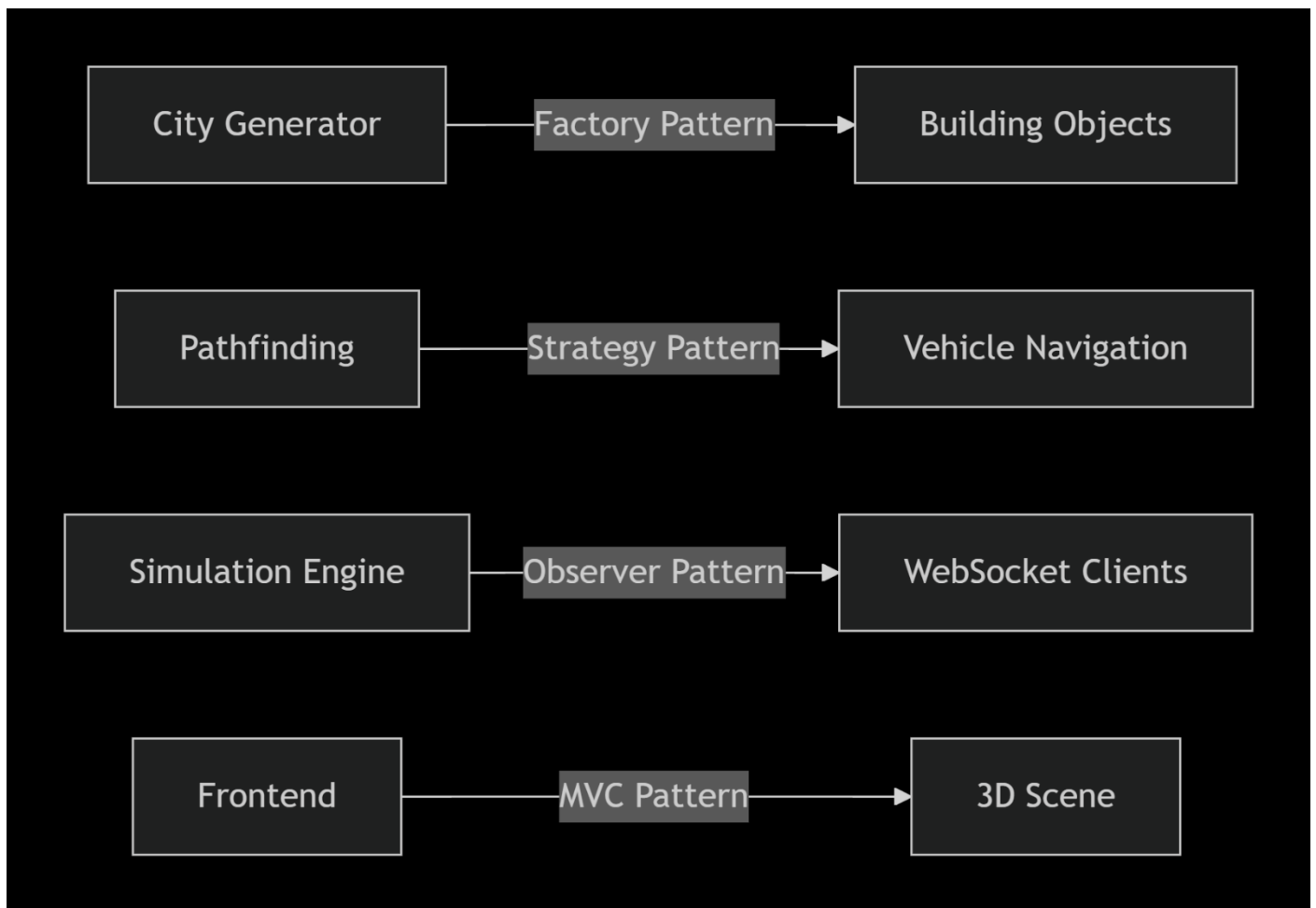
3D Rendering

- Three.js: 3D rendering library for the digital twin visualization
- Orbit Controls: Camera controls for user interaction

Modern JavaScript Features

- ES Modules
- Fetch API: HTTP requests to the backend
- WebSocket API: Real-time data streaming

5. Design Patterns



Factory Pattern

- `make_building()`: Creates standardized building objects with properties
- `spawn_vehicle()`: Generates vehicle entities with consistent structure

Strategy Pattern

- `pick_next()`: Navigation strategy for vehicles
- `update_traffic()`: Contains multiple traffic management strategies

Observer Pattern

- **WebSocket connection:** Real-time state updates to connected clients
- **Frontend state management:** React to simulation changes

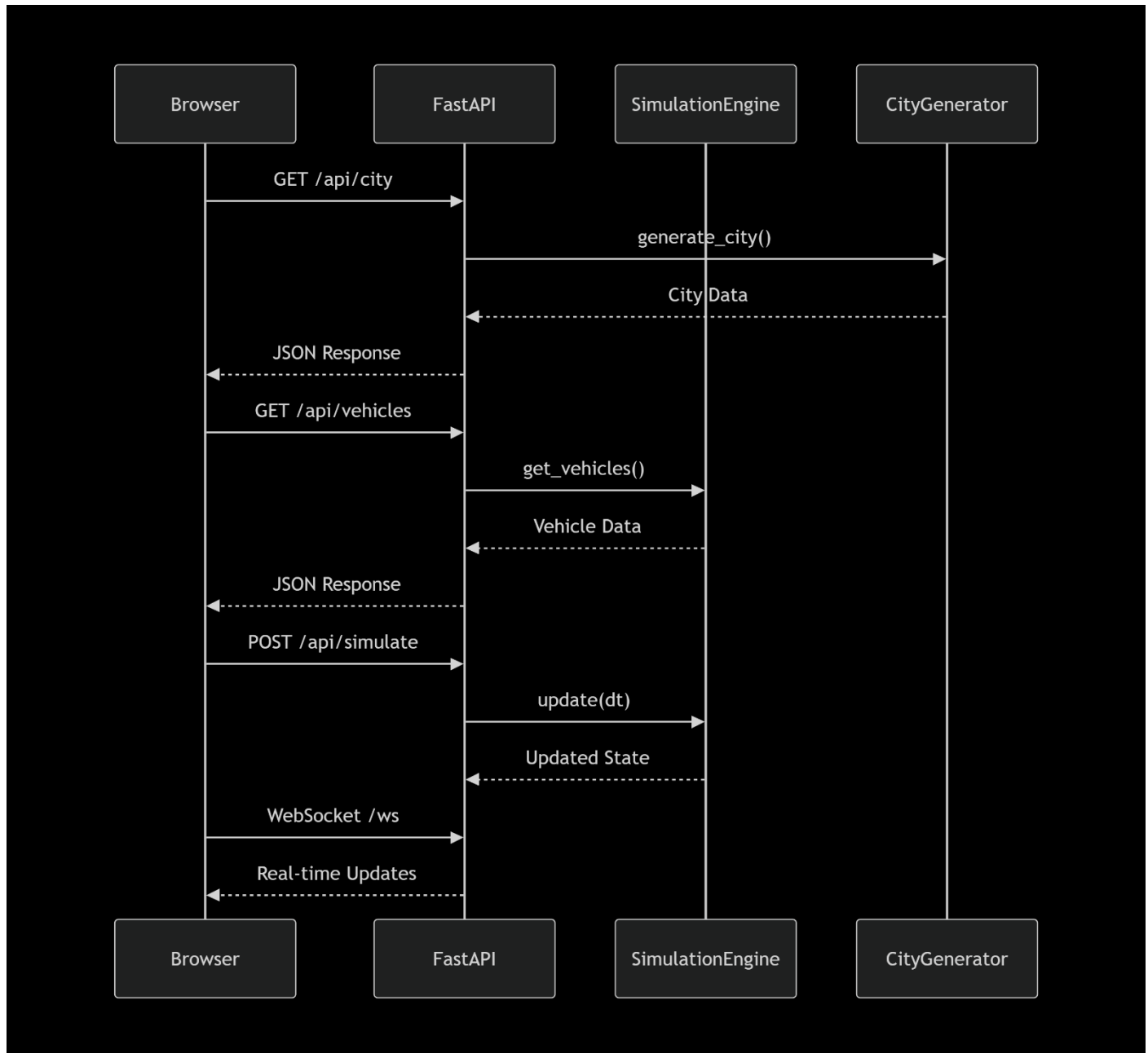
Singleton Pattern

- **SimulationEngine:** Single instance managing all simulation state

Model-View-Controller

- Model: Pydantic schemas and simulation state
- View: Three.js 3D visualization
- Controller: FastAPI endpoints and WebSocket handlers

6. Data Flow Diagram



7. Key Technical Decisions

City Generation Algorithm

- Grid-based city generation: 6×6 grid with standardized blocks
- Procedural building generation: Randomly generated building types and properties
- Zoning: Different building types (residential, commercial, office, emergency services)

Pathfinding & Traffic Simulation

- Waypoint-based navigation: Vehicles travel between intersection nodes
- Intelligent traffic lights: Cycle-based traffic management
- Car-following model: Vehicles maintain safe distances based on speed and gaps

3D Visualization Approach

- Procedural textures: Window patterns generated on canvas
- Real-time updates: Vehicles move smoothly based on simulation state
- Interactive features: Click to select and follow entities

8. API Endpoints Structure

/api/health - Health check endpoint

/api/city - Get static city data (buildings, roads, intersections)

/api/vehicles - Get current vehicle states

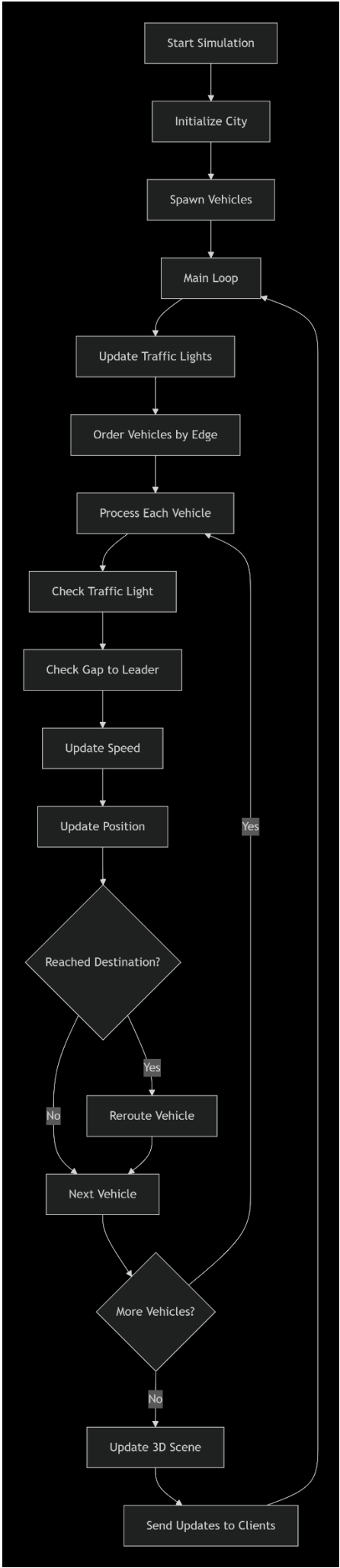
/api/vehicles/{id}- Get specific vehicle details

/api/snapshot - Full simulation snapshot

/api/simulate - Advance simulation by one step

/ws - WebSocket for real-time updates

9. Simulation Loop Flowchart



10. Testing Strategy

